SC2002 Object-Oriented Design and Programming
Group Assignment Report · AY2026/27 Semester 1

Restaurant Rush

A turn-based restaurant management game for the command line, written in Java and built in three tagged stages.



Lab group	SCMA, Group 3
Members	CCDS LEE SHENG KAI CCDS LEONG XUAN HAO CCDS POLOYAPOY RON JOSHUA ABES (Apollo) CCDS VICTORIA HAUGAA
Repository	https://github.com/theApolloAlien/restaurant-rush
Stage tags	`stage-1` , `stage-2` , `stage-3`
Build	Java 17 or newer, no external libraries. Run `./run.sh` (or `run.bat` on Windows).

Contents

1 Overview 3

2 UML class diagram 4

3 UML sequence diagram 6

4 Design discussion 13

5 How the player's choices change the result 16

6 Development process and stage comparisons 17

7 Functional evidence 19

8 Rule interpretations 26

9 Reflection and contributions 27

1 Overview

In Restaurant Rush the player is the restaurant manager. Each turn they tell the Host, Waiter and Chef what to do, and the Cashier collects payment automatically. With one of each and only two tables, there is never enough staff time, so every turn comes down to the same question: who do we look after first?

BASE SETTING

- 18 turns, 2 tables, no early ending
- Waiter, Chef and Cashier, one task each per turn (Stage 3 adds a Host)
- One arrival on turns 1, 4, 7, 10, 13 and 16
- Salad $9 (1 unit), Burger $15 (1), Pasta $18 (2)
- Win: at least 4 paid, $45 revenue and an average satisfaction of 60

RUNNING IT

```
./run.sh              # build and play
./run.sh --script scripts/stage3-victory.txt
./run.sh --plain      # no colour, ASCII only
./test.sh             # 22 rule checks
git checkout stage-1  # then ./build.sh
```

After checking out a tag, run `./build.sh` before `./run.sh`, because the jar is not tracked by git. On Windows, `run.bat` and `test.bat` always recompile. Only a JDK is needed, with no build tool and no network access.

What happens in one turn

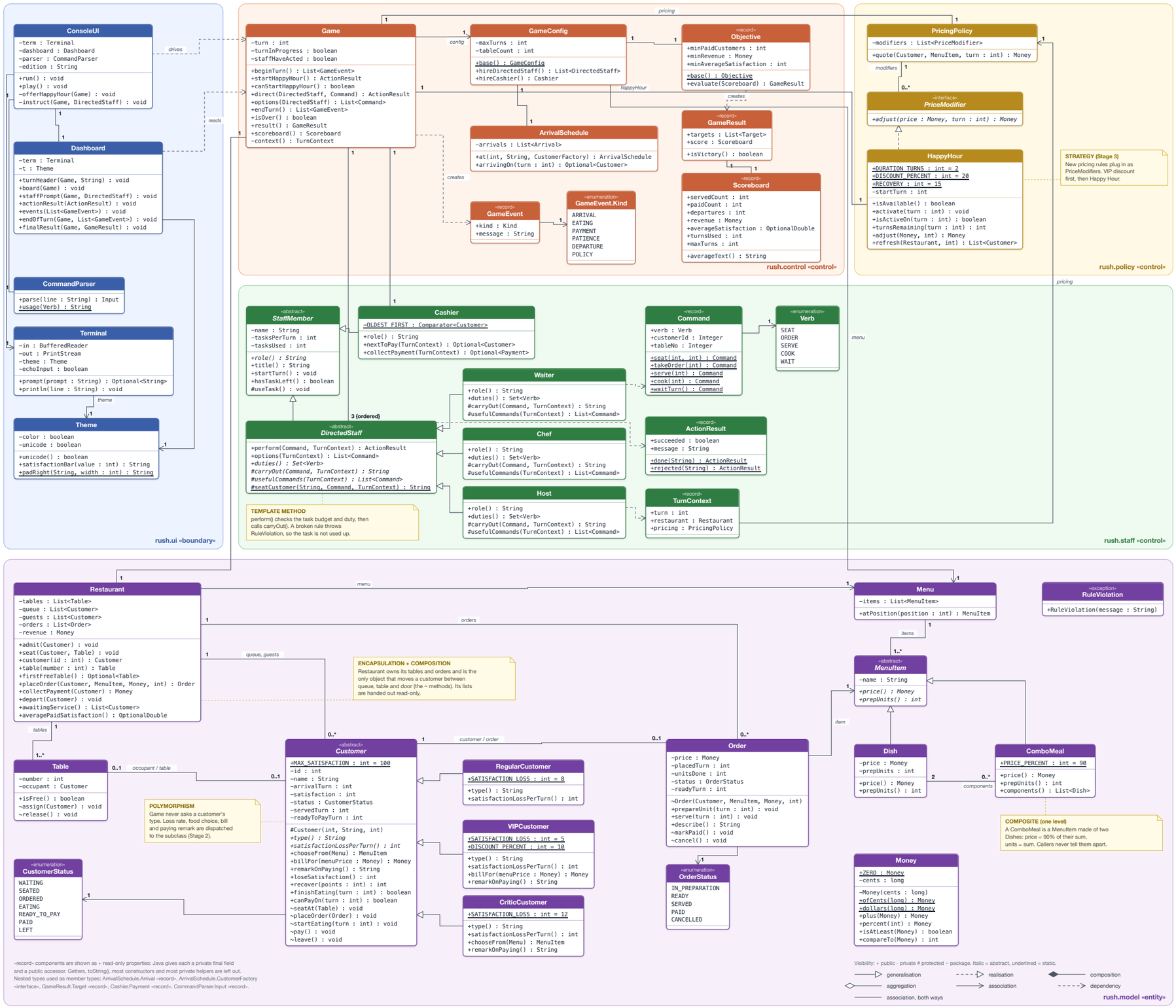
Step	What happens	Where the rule lives
0	Start-of-turn display	`Dashboard` shows the turn, revenue, average, queue, tables, satisfaction, order status, available staff tasks and Happy Hour status
1	Arrivals	`Game.beginTurn` asks `ArrivalSchedule.arrivingOn(turn)`, then calls `Restaurant.admit`
2	Manager choices	`ConsoleUI` reads one command per staff member; `DirectedStaff.perform` checks it (Host, then Waiter, then Chef)
3	Kitchen and serving	`Order.prepareUnit` marks the order READY on its last unit; `Order.serve` only accepts a dish that was READY before this turn
4	Eating and payment	`Customer.finishEating`, then `Cashier.collectPayment` (one per turn, oldest first)
5	Waiting and abandonment	`Customer.loseSatisfaction` (rate depends on the customer type) and `Restaurant.depart` at 0
6	Result after turn 18	`Objective.evaluate(Scoreboard)` returns a `GameResult`

The three stages

Tag	Adds	Base-setting customers	Rule checks
`stage-1`	Regular customers, menu, Waiter, Chef, Cashier, full turn sequence, both endings	6 Regular	12
`stage-2`	`VIPCustomer` and `CriticCustomer`	Critic on turn 4, VIP on turn 7, 4 Regular	16
`stage-3`	`Host`, `ComboMeal` and Happy Hour	Same as Stage 2	22

2 UML class diagram (final design, Stage 3)

Drawn from the code at tag **`stage-3`**. Getters, `toString()`, most constructors and most private helpers are left out, and «record» components are shown as read-only properties. Editable version: `docs/diagrams/01-class-diagram.drawio`.



2.1 Reading the class diagram

ENCAPSULATION

`Money` is immutable and stores integer cents, so prices add up exactly and each discount is rounded once. `Restaurant` is the only class that moves a customer between the queue, a table and the door: `Customer.seatAt`, `placeOrder`, `startEating`, `pay` and `leave`, and `Table.assign` and `release`, are package-private (`~`). The few public methods that change state (`finishEating`, `loseSatisfaction`, `recover`, `Order.prepareUnit` and `serve`) check the current status first. Every list is handed out read-only, and the UI only gives commands to `Game`.

POLYMORPHISM

`Customer` owns the visit life cycle. The subclasses only override `satisfactionLossPerTurn()`, `chooseFrom(Menu)`, `billFor(Money)` and `remarkOnPaying()`, plus the `type()` label, and `Game`, `Waiter` and `Cashier` never check a customer's type. In the same way, `Dish` and `ComboMeal` answer `price()` and `prepUnits()` differently, while `Order` and `Chef` treat them the same.

COMPOSITION AND AGGREGATION

`Game` owns its restaurant, staff, Happy Hour and pricing policy, which exist only for one game. `Restaurant` owns its two tables and its orders. Customers arrive from outside, so `Restaurant` only aggregates them. A `ComboMeal` aggregates two `Dish` objects that are also on the menu by themselves.

INHERITANCE

`StaffMember` holds the per-turn task budget. `DirectedStaff` adds the template method `perform()`, and `Waiter`, `Chef` and `Host` fill in `duties()` and `carryOut()`. The Cashier takes no instructions, so it extends `StaffMember` directly. `HappyHour` implements the `PriceModifier` interface.

The packages form layers, and dependencies only point one way: **ui** reads input and draws the screen, **control** runs the turn sequence, **staff** decides what each role may do, **policy** holds the pricing rules, and **model** holds the entities and their rules. The small records (`Command`, `ActionResult`, `TurnContext`, `Objective`, `Scoreboard`, `GameResult`, `GameEvent`) are immutable value objects.

3 UML sequence diagram

The scenario comes from the Stage 3 victory run (`scripts/stage3-victory.txt`). It follows **VIP customer #3 Chloe** from her arrival on turn 7 to her payment on turn 11, and then the result check after turn 18. It is drawn in six parts so that it stays readable when printed. The message numbers carry on from one part to the next, and the table below refers to them.

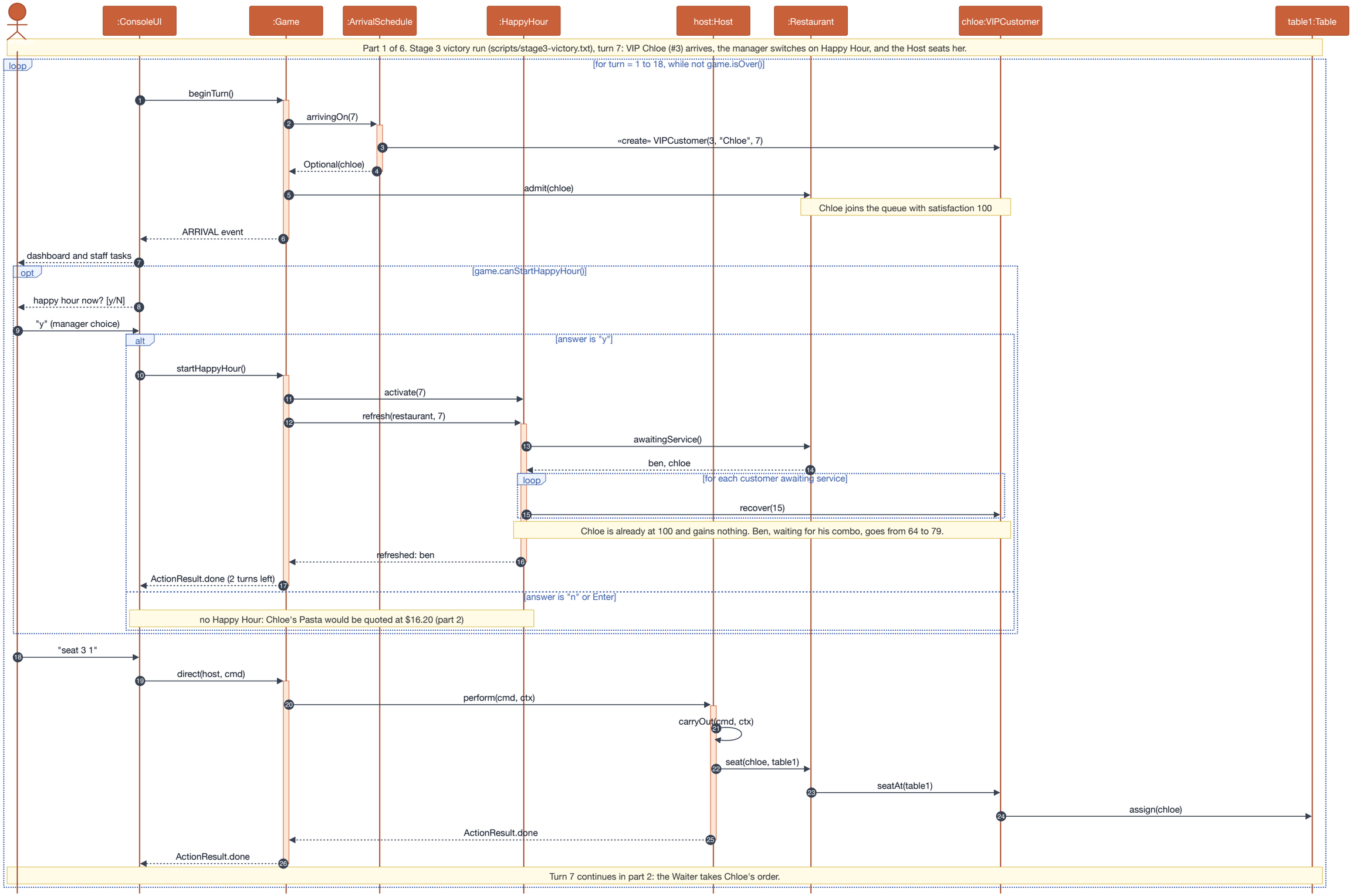
Required element	Where to find it
Customer arrival and order	Part 1, messages 1–7 (`arrivingOn` , «create», `admit`); part 2, 27–44 (`chooseFrom` , `quote` , `placeOrder` , «create» `Order`)
Waiter and Chef assignments	Host 18–26; Waiter 27–44 and 70–82; Chef 45–51 and 60–66. Every instruction goes through `Game.direct` and the template method `perform` , which calls `carryOut`
READY then SERVED	Part 3: `prepareUnit(8)` makes the order READY (64). Part 4: `serve(9)` (75) succeeds only if `status == READY and readyTurn < turn` , otherwise it is rejected and the task is not used (76–82)
Eating and payment	Part 5: `finishEating(10)` (84). On turn 10 the Cashier charges Ben, who has been ready longer (86–90). On turn 11: `collectPayment` , `pay()` , `markPaid()` , `release()` and the revenue update (92–106)
Result check	Part 6, 107–117: `result()` , `Objective.evaluate` , `GameResult.isVictory()` , then the Victory or Defeat screen
Stage 2 and 3 behaviour	VIP pricing (`billFor` , 10% off, 35–36); Happy Hour (`activate` , `refresh` , `recover` and `adjust` , 10–17, 37–38 and 56–58); the Host (18–26)
Guards and loops	`loop` over turns, waiting customers, price modifiers, guests and the Waiter's retries; `opt [game.canStartHappyHour()]` ; `alt` for the manager's Happy Hour answer, a serve that succeeds or is rejected, and Victory or Defeat; `opt` for the Cashier

The manager choice that changes the outcome

On turn 7 the manager answers `y` to Happy Hour (message 9) before the Waiter takes Chloe's order. `PricingPolicy.quote` applies the VIP discount first and then Happy Hour (34–39), so her Pasta is locked at $18.00 × 0.9 × 0.8 = **$12.96** instead of $16.20, and the waiting Critic recovers 15 points on turns 7 and 8. Over the whole run this costs $3.24 in revenue but raises the final average satisfaction from 75.6 to **82.6** (see Section 5).

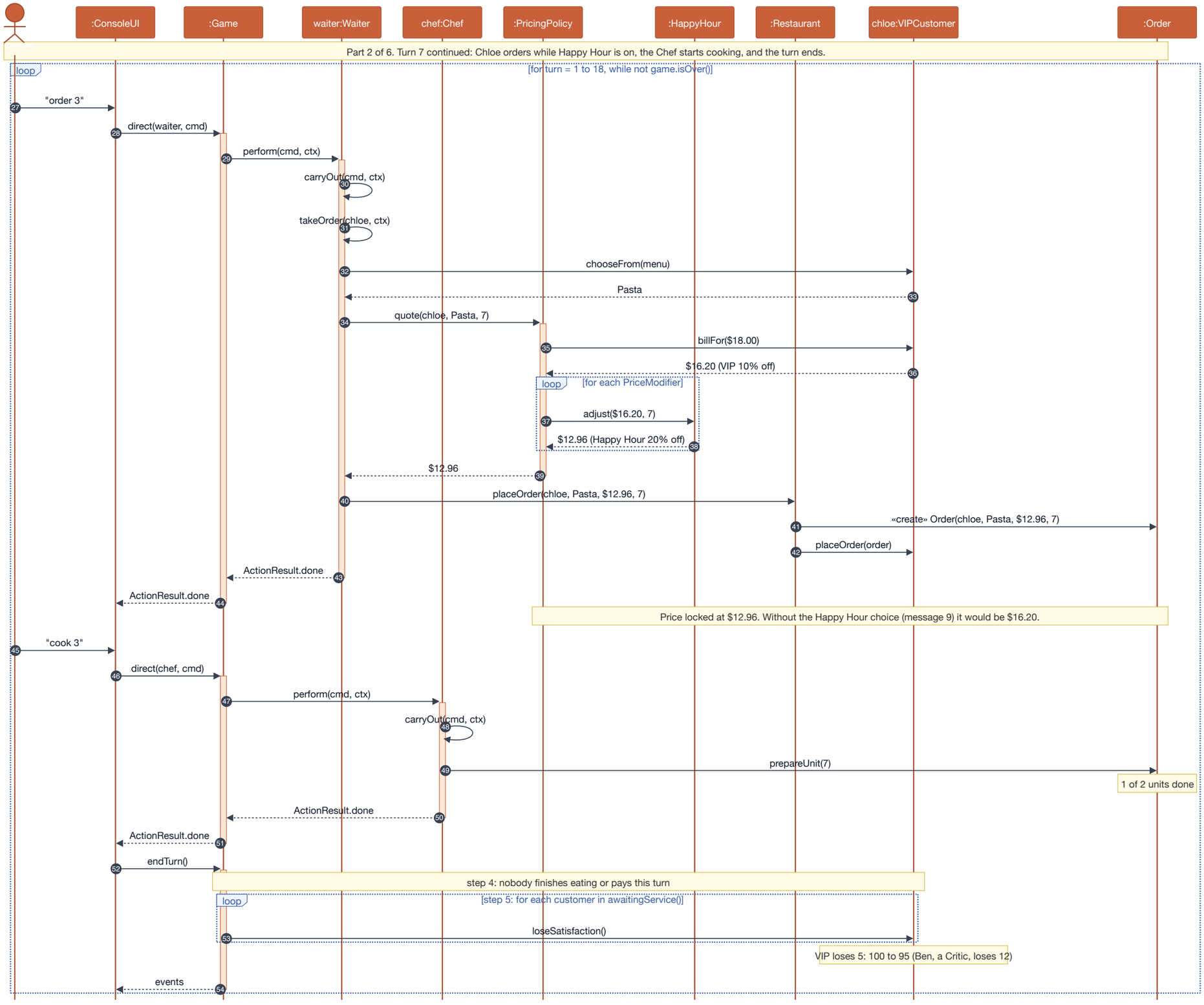
Sequence diagram, part 1. Turn 7: arrival, Happy Hour and seating (messages 1–26)

Editable version: `docs/diagrams/02-sequence-part-1-arrival-seating.drawio`



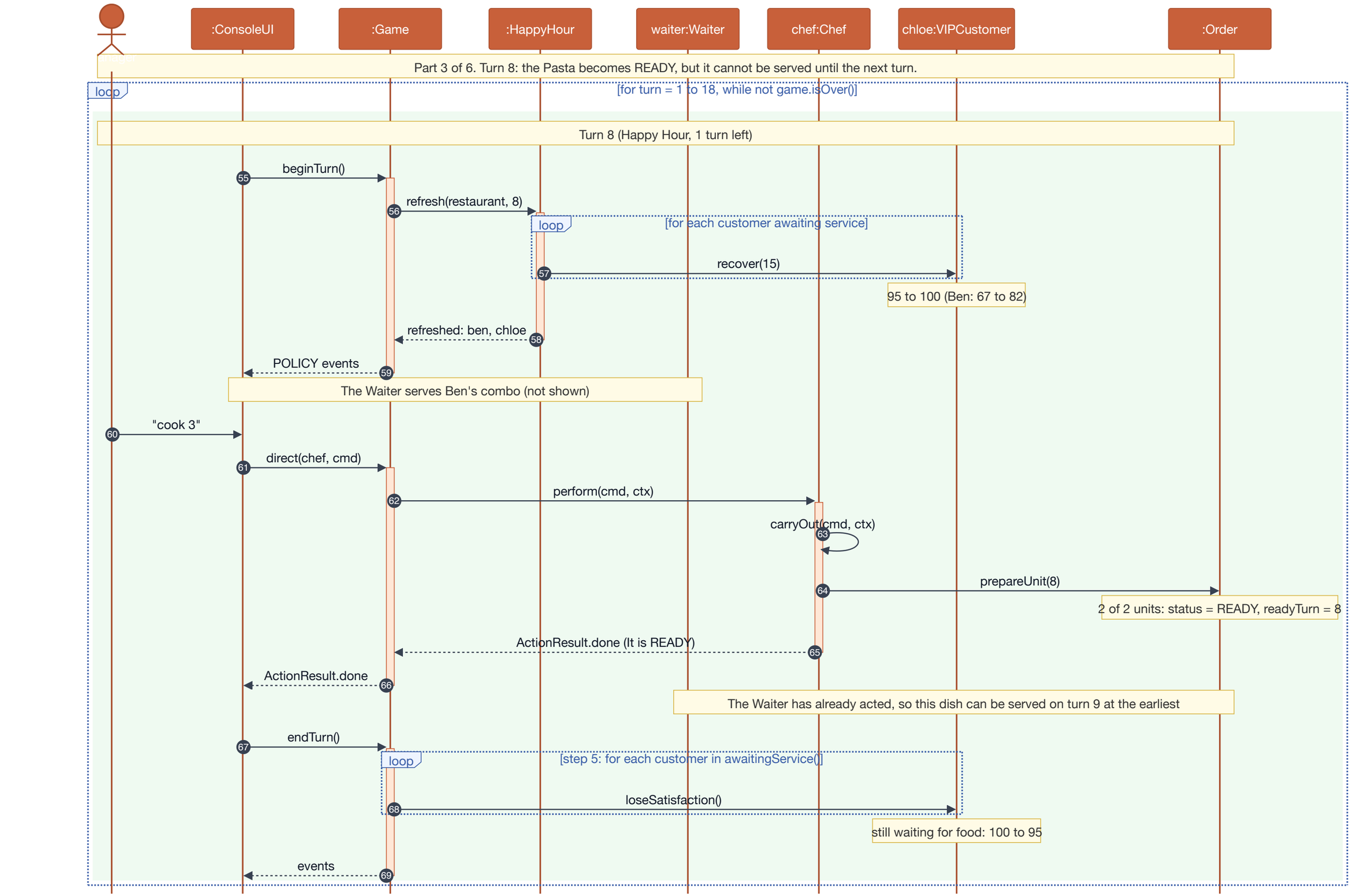
Sequence diagram, part 2. Turn 7: ordering, pricing and cooking (messages 27–54)

Editable version: `docs/diagrams/03-sequence-part-2-order-pricing.drawio`



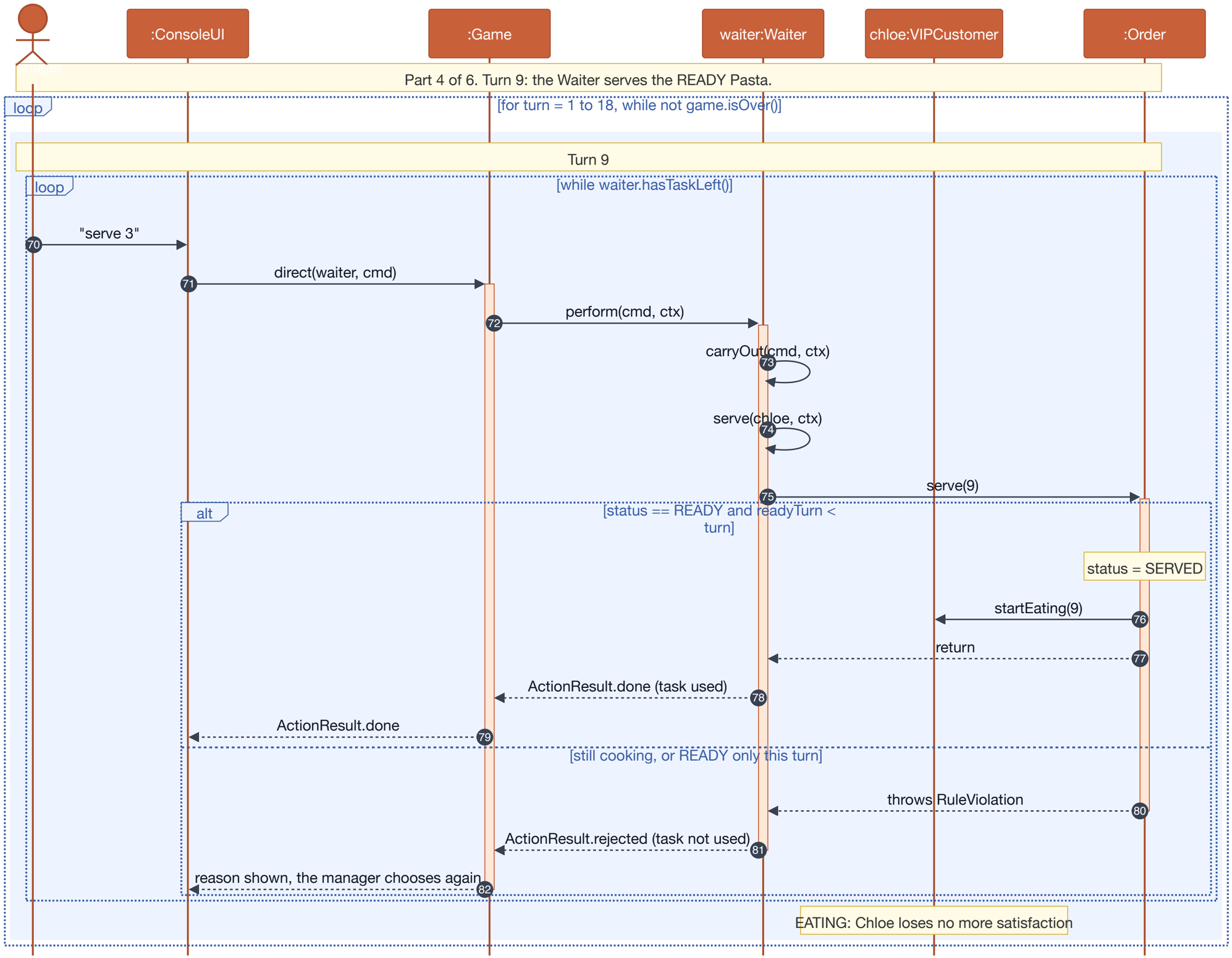
Sequence diagram, part 3. Turn 8: the order becomes READY (messages 55–69)

Editable version: `docs/diagrams/04-sequence-part-3-ready.drawio`



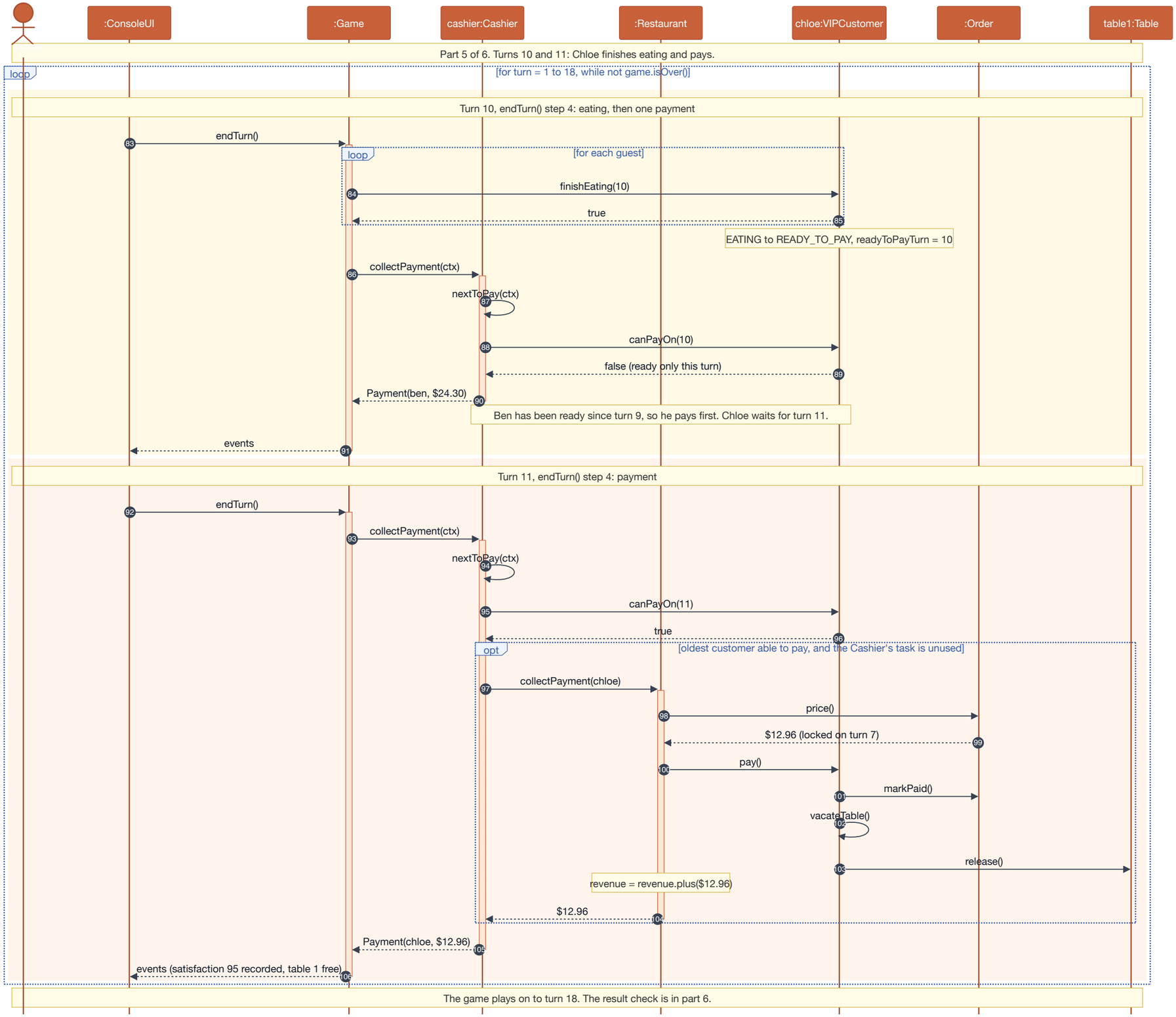
Sequence diagram, part 4. Turn 9: the order is SERVED (messages 70–82)

Editable version: `docs/diagrams/05-sequence-part-4-served.drawio`



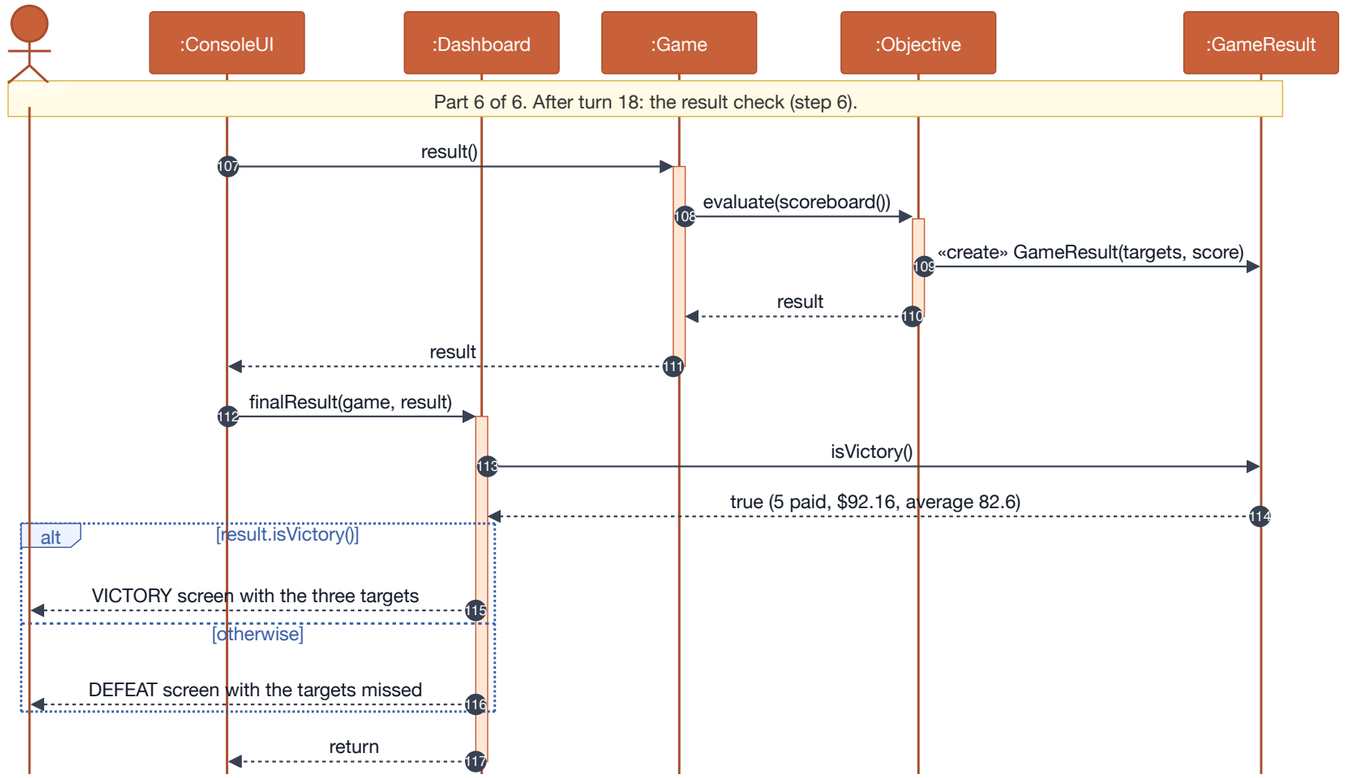
Sequence diagram, part 5. Turns 10 and 11: eating and payment (messages 83–106)

Editable version: `docs/diagrams/06-sequence-part-5-eating-payment.drawio`



Sequence diagram, part 6. After turn 18: the result check (messages 107–117)

Editable version: `docs/diagrams/07-sequence-part-6-result.drawio`



4 Design discussion

4.1 Key abstractions and their responsibilities

Abstraction	Layer	Responsibility
`Game`	control	Runs the six steps of a turn in order. The rules it triggers live in domain and staff objects.
`GameConfig`, `ArrivalSchedule`	control	The base setting: turns, tables, menu, who arrives when and which staff are hired. Each stage registers its additions here.
`Objective`, `Scoreboard`, `GameResult`	control	The targets, the end-of-turn figures, and the Victory or Defeat verdict.
`StaffMember`, `DirectedStaff`	staff	The per-turn task budget, and the template method that checks and carries out an instruction.
`Waiter`, `Chef`, `Host`	staff	What one role may do, and which commands are useful right now (shown as hints).
`Cashier`	staff	Takes one payment per turn from the customer who has been ready longest.
`PricingPolicy`, `PriceModifier`, `HappyHour`	policy	Locks the bill when an order is taken. Happy Hour's once-per-game life cycle, discount and recovery.
`Restaurant`	model	Keeps the queue, tables, guests, orders and revenue consistent with each other.
`Customer` and 3 subclasses	model	The visit life cycle and satisfaction (base class); patience, food choice and bill (subclasses).
`Order`	model	The locked price, cooking progress, the READY then SERVED rule, and cancellation.
`MenuItem`, `Dish`, `ComboMeal`, `Menu`	model	What can be ordered. A combo works out its price and units from its dishes.
`Money`, `RuleViolation`	model	Exact cent arithmetic, and a readable reason when a move breaks a rule.
`ConsoleUI`, `Dashboard`, `CommandParser`, `Terminal`, `Theme`	boundary	Prompts, drawing the state, checking the shape of a command, input/output, and colours with an ASCII fallback.

4.2 Design principles in the code

Single responsibility. Each rule lives in one place. "A dish finished this turn is served next turn at the earliest" is only checked in `Order.serve`. The Waiter just calls it, and the UI just prints the `RuleViolation` message. `CommandParser` only checks that `seat 2 1` is well formed, never whether the move is legal, and `Dashboard` only reads the game.

Open/closed. Going from Stage 1 to Stage 2 added the VIP and Critic without touching `Game`, `Customer`, `Restaurant`, `Order` or any staff class (`git diff stage-1 stage-2 -- src/rush/control/Game.java src/rush/staff` is empty). In Stage 3 the Host is a new `DirectedStaff` registered in `GameConfig`, and the console's loop over `game.directedStaff()` picks it up unchanged (`ConsoleUI` changed only for the

Happy Hour prompt and auto-wait, see Section 6). Combos needed no change to `Order`, `Chef` or `Waiter`.

Liskov substitution. The status transitions (`seatAt`, `placeOrder`, `pay`, `leave`) are written once in `Customer` and no subclass overrides them. The overrides keep the base contract: a loss rate of at least 0, an item from the given menu, and a bill no higher than the menu price. A rule check runs a VIP through exactly the same flow as a Regular. A `ComboMeal` can stand in for any `MenuItem`, and the Critic's choice and the Chef's unit counting work unchanged.

Interface segregation. The Cashier takes no instructions, so it is a `StaffMember` but not a `DirectedStaff`, and it does not have to implement `duties()` or `carryOut()`. `ConsoleUI` only uses the small `DirectedStaff` API, and `PriceModifier` has a single method.

Dependency inversion (partly). `PricingPolicy` depends on `PriceModifier` rather than `HappyHour`, and `ArrivalSchedule` depends on a `CustomerFactory` interface, so it never names a concrete customer class. However, `Game` still creates `HappyHour` and `PricingPolicy` itself. Passing them in through `GameConfig` would be the next step if more policies were added.

4.3 Patterns we used

Pattern	Where	Why
Template method	`DirectedStaff.perform`	The task-budget check, the duty check and "a rejected move costs nothing" are written once for every role.
Composite (one level)	`MenuItem`, `Dish`, `ComboMeal`	A combo is a menu item built from two `Dish` leaves, so its price and units are derived rather than copied. A combo cannot contain another combo.
Strategy	`PricingPolicy` with `PriceModifier`	Restaurant-wide pricing rules plug in after the customer's own discount, in a fixed order.
Command (simple)	`Command` record	Separates what the manager typed from who checks it, so a script is just a list of commands.
Factory references	`ArrivalSchedule.at(4, "Ben", CriticCustomer::new)`	Each stage registers its customers without the schedule knowing the customer classes.

4.4 Trade-offs

Decision	Benefit	Cost
Domain objects throw `RuleViolation` with a message for the player	One source of truth per rule, and the message shown is the rule itself. The UI never repeats a "can I?" check.	Exceptions on an expected path. Only `DirectedStaff.perform` and `Game.startHappyHour` catch them.
`Customer` abstract from Stage 1, `MenuItem` concrete until Stage 3	Customer types were certain to come. Combos were the only reason for a second kind of menu item, so we waited until we needed it.	Stage 3 had to change `MenuItem` and its setup in `GameConfig`. The callers did not change.
Events are short English strings	Simple, and the UI styles them by `GameEvent.Kind` without parsing.	Tests have to compare text. A structured event per kind would be cleaner.
The dashboard colours customer types by name	Presentation stays out of the domain.	There is a small switch on `type()` in the UI, used only for colour. The rules have no type switch.
Staff with nothing useful to do wait automatically (Stage 3)	Fewer pointless prompts once there are three roles per turn.	A UI change between stages (documented in Section 6).

4.5 An alternative we considered: the State pattern for customers

A customer moves through `WAITING → SEATED → ORDERED → EATING → READY_TO_PAY → PAID`, or `LEFT`. We sketched one class per state, each implementing `seat()`, `order()`, `serve()` and `endOfTurn()` and rejecting moves it does not allow. That puts every transition in one place per state. We kept the enum because the life cycle is strictly linear, and the type behaviours (loss rate, bill, food choice) cut across every state. We would have had seven state classes and three customer types to keep in sync. An enum with guarded methods in one abstract class is shorter, easy to test, and keeps the polymorphism in the customer types, where the brief asks for it.

We also rejected a `CustomerType` enum with switches in the controller, because the brief asks for dispatch through the customer objects. We also rejected a Decorator chain for prices, because "VIP first, then Happy Hour" reads more clearly as an ordered list in `PricingPolicy`.

5 How the player's choices change the result

The base setting has no randomness, so any difference in the outcome comes from the manager. We replayed the real `Game` with fixed Waiter priorities, with and without the Chef cooking an order on the turn it is taken, and in Stage 3 with and without the Host and with different Happy Hour turns.

Stage 1

Waiter priority	Chef cooks same turn	Paid	Revenue	Avg sat.	Result
serve, order, seat	yes	5	$66.00	79.2	**Victory**
serve, order, seat	no	4	$51.00	66.0	**Victory**
seat, order, serve	no	4	$51.00	60.0	**Victory (just)**
seat, serve, order	no	3	$42.00	60.0	**Defeat**

Stage 2 (Critic and VIP)

Waiter priority	Chef cooks same turn	Paid	Revenue	Avg sat.	Result
serve, order, seat	yes	5	$67.20	76.0	**Victory**
seat, order, serve	no	4	$52.20	62.5	**Victory**
seat, serve, order	no	3	$43.20	59.3	**Defeat on all three targets**

Stage 3 (Host, combos, Happy Hour)

Waiter priority	Chef same turn	Host used	Happy Hour	Paid	Revenue	Avg	Result
order, serve, seat	yes	yes	turn 7	5	$92.16	82.6	**Victory**
order, serve, seat	yes	yes	never	5	$95.40	75.6	**Victory**
serve, order, seat	yes	yes	turn 7	4	$67.86	84.3	**Victory**
seat, serve, order	no	yes	turn 7	4	$67.86	81.0	**Victory**
seat, serve, order	no	no	turn 7	3	$49.50	68.7	**Defeat**
seat, serve, order	no	no	turn 14	3	$49.50	57.0	**Defeat**

Three things stand out:

- **Serve before you seat.** A served customer stops losing satisfaction and frees the table two turns later. Seating first only moves the waiting from the queue to a table.
- **The Host changes the best priority.** Once the Host does the seating, taking orders first beats serving first (5 paid against 4). Using the Host alone turns the weakest strategy from a defeat into a victory.
- **Happy Hour is a trade-off.** On turn 7 it costs $3.24 in revenue but lifts the average from 75.6 to 82.6. On turn 14 it helps nobody who can still pay before the game ends.

6 Development process and stage comparisons

Each stage is a tagged commit that runs on its own. We checked every tag from a clean `git archive` copy: it compiles, passes its own rule checks, and replays its own victory and defeat scripts.

Commit history

Commit	Date	Message	Tag
`7727982`	2026-10-08	Stage 1: domain model (Money, menu, customers, orders, tables, restaurant)	
`a724465`	2026-10-08	Stage 1: staff roles and the six-step turn controller	
`3a27533`	2026-10-08	Stage 1: command-line interface with dashboard and command parser	
`ebdc913`	2026-10-08	Stage 1: rule checks, reproducible victory/defeat scripts and test notes	`stage-1`
`a721c2d`	2026-10-08	Stage 2: VIP and Critic customers dispatched through Customer subclasses	`stage-2`
`13d6dc5`	2026-10-08	Stage 3: Host, ComboMeal (MenuItem becomes abstract), Happy Hour pricing policy	`stage-3`

The table runs up to the `stage-3` tag. Later commits only add the report, diagrams and submission files: `git diff stage-3 main -- src test scripts` is empty. Compare on GitHub: stage-1...stage-2 and stage-2...stage-3. Locally: `git diff stage-1 stage-2 --stat`.

Stage 1 to Stage 2: customer behaviour (commit `a721c2d`)

File	Change	Reason
`model/VIPCustomer.java`	new	Loses 5 per turn; `billFor` gives 10% off.
`model/CriticCustomer.java`	new	Loses 12 per turn; overrides `chooseFrom` to order the most elaborate dish, and leaves a star review.
`model/Money.java`	+5 lines	`percent(int)`, rounded half-up, first needed for the VIP discount.
`control/GameConfig.java`	small	Registration (two arrival lines and their imports): Ben (turn 4) becomes a Critic and Chloe (turn 7) a VIP. Still six arrivals.
`ui/Dashboard.java`	small	A colour per customer type and a legend in the help screen (presentation only).
`Main.java`, tests, scripts, docs	updated	Edition label, 4 new rule checks, Stage 2 scripts and notes.
`Game`, `Customer`, all staff, `Restaurant`, `Order`	unchanged	The new behaviour is dispatched through the new subclasses.

Stage 2 to Stage 3: new role and menu policy (commit `13d6dc5`)

File	Change	Reason
`staff/Host.java`	new	A `DirectedStaff` whose only duty is `SEAT`. It reuses the shared seating helper.
`model/Dish.java`, `model/ComboMeal.java`	new	The concrete dish moved into `Dish`. `ComboMeal` takes 90% of the price and the sum of the units of its two dishes.
`model/MenuItem.java`	rewritten	Changed from concrete to abstract (`price()`, `prepUnits()`). Its public methods are the same, so `Order`, `Chef`, `Waiter` and the Critic did not change. Only the setup in `GameConfig` now builds `Dish`es.
`policy/PriceModifier.java`, `policy/HappyHour.java`	new	Once per game, two turns, 20% off orders taken, and 15 points of recovery at turn start for customers still waiting.
`policy/PricingPolicy.java`	changed	Applies a list of `PriceModifier`s after `billFor`, so the VIP discount comes first and Happy Hour second.
`model/Customer.java`	+14 lines	`recover(points)`, capped at 100, only for customers still waiting for service.
`control/Game.java`	changed	Owns `HappyHour`. `startHappyHour()` only works before any staff member acts, and recovery happens in `beginTurn()`.
`control/GameConfig.java`	small	Registration: the menu is built from `Dish`es plus two combos, and the Host is hired first.
`control/GameEvent.java`	1 line	A `POLICY` event kind for Happy Hour recovery.
`ui/ConsoleUI.java`, `ui/Dashboard.java`	changed	The Happy Hour prompt and remaining-duration line, and auto-wait for staff with nothing useful to do.
`Main.java`, tests, scripts, docs	updated	Edition label, 6 new rule checks (22 in total), Stage 3 scripts and notes.
`Waiter`, `Chef`, `Cashier`, `DirectedStaff`, `Restaurant`, `Order`	unchanged	The Host and combos fit the existing abstractions.

Earlier behaviour still works. The Stage 3 rule checks still cover the Stage 1 worked example (Pasta from arrival to payment, satisfaction 92, 84, 76, paid $18.00 on turn 6), abandonment, the Cashier's order and the "READY, then SERVED next turn" rule, and they all pass.

7 Functional evidence

Every screen below is real output, captured by replaying the scripts at their own tag. Full plain-text transcripts are in `docs/transcripts/`.

7.1 Launch, dashboard and input handling (Stage 1, turn 1)

The start-of-turn display shows the turn, revenue against the target, paid count, average satisfaction (`N/A` before anyone pays), the queue, both tables, each customer's satisfaction, the kitchen and the tasks each staff member can do. Two bad inputs are explained and do not use up the Waiter's task.

Stage 1: launch, first dashboard and two rejected inputs (turn 1)

```
RESTAURANT RUSH
a turn-based restaurant management game · Stage 1 · Basic operation

1  Play the base setting
2  How to play
3  Quit

› 1

TURN 1 of 18  ▰▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱  Stage 1 · Basic operation
Revenue $0.00/$45.00   Paid 0/4   Avg satisfaction N/A/60   Left 0

  ▸ #1 Ada (Regular) arrives and joins the queue with satisfaction 100.

WAITING QUEUE
   #1 Ada    Regular  ▰▰▰▰▰▰▰▰ 100  Waiting for a table since turn 1

TABLES
 T1  free
 T2  free

KITCHEN
   no orders yet

STAFF TASKS AVAILABLE
   Waiter Wendy      seat 1 1  ·  wait
   Chef Carlo        wait
   Cashier Cassie    automatic · nobody is ready to pay yet

◆ WAITER WENDY  seat, order, serve, wait
  try seat 1 1  ·  wait
waiter › order 1
  ✘ #1 Ada has no table yet. Seat them first.
    Nothing was used up. Choose again.
waiter › dance
  ✘ I don't know the command 'dance'. Type 'help' for the list.
waiter › seat 1 1
  ✔ Wendy seats #1 Ada at table 1.

◆ CHEF CARLO  cook, wait
  try wait
chef › wait
  ✔ Chef Carlo waits this turn.
```

7.2 READY to SERVED, and a decision under pressure (Stage 1, turns 9 and 10)

On turn 9, `serve 3` is rejected because the Pasta is only half cooked. The Chef then finishes it, but the Waiter has already acted. On turn 10, Dev arrives just as the Pasta is READY. There is only one Waiter, so the manager serves Chloe and leaves Dev in the queue for a turn.

Stage 1, turn 9: serve rejected (still cooking), then the Chef makes it READY

```
◆ WAITER WENDY  seat, order, serve, wait
  try wait
waiter › serve 3
  ✘ #3 Chloe's Pasta is still cooking (1/2 units done).
    Nothing was used up. Choose again.
waiter › wait
  ✔ Waiter Wendy waits this turn.

◆ CHEF CARLO  cook, wait
  try cook 3  ·  wait
chef › cook 3
  ✔ Carlo finishes #3 Chloe's Pasta (2/2). It is READY.

─── End of turn 9 ──────────────────────────────────────────
  ▾ #3 Chloe waiting for pasta: satisfaction 84 -> 76 (-8)

Served 2 · Unhappy departures 0 · Revenue $24.00 · Avg sat. 84.0 · Turns 9/18
```

Stage 1, turn 10: Dev arrives as the Pasta is READY; one Waiter, two needs

```
┌──────────────────────────────────────────────────────────┐
│ TURN 10 of 18  ▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱   Stage 1 · Basic operation │
├──────────────────────────────────────────────────────────┤
│ Revenue $24.00/$45.00   Paid 2/4   Avg satisfaction 84.0/60   Left 0 │
└──────────────────────────────────────────────────────────┘

  ▸ #4 Dev (Regular) arrives and joins the queue with satisfaction 100.

WAITING QUEUE ─────────────────────────────────────────────
   #4 Dev    Regular  ▰▰▰▰▰▰▰▰▰▰ 100  Waiting for a table since turn 10

TABLES ────────────────────────────────────────────────────
 T1  #3 Chloe  Regular  ▰▰▰▰▰▰▰▰▰▰
```

7.3 Payment, victory and defeat (Stage 1)

Chloe was served on turn 10, finished eating on turn 11 and pays on turn 12, which frees table 1.

Stage 1: end of turn 12, the Cashier collects Chloe's Pasta

```
End of turn 12
$ Cassie collects $18.00 from #3 Chloe (satisfaction 76 recorded). Table 1
  is free. "Not bad at all."
▾ #4 Dev waiting for salad: satisfaction 84 -> 76 (-8)

Served 3 · Unhappy departures 0 · Revenue $42.00 · Avg sat. 81.3 · Turns 12/18
```

Stage 1 victory run: result screen

```
VICTORY
The dining room hums. Every target was met.

TARGETS
  ✔ Paid customers                5   at least 4
  ✔ Revenue                  $66.00   at least $45.00
  ✔ Average satisfaction       79.2   at least 60

GUESTS
  Guest       Type      Ordered              Bill  Sat.  Outcome
  #1 Ada      Regular   Salad               $9.00    84  Paid
  #2 Ben      Regular   Burger             $15.00    84  Paid
  #3 Chloe    Regular   Pasta              $18.00    76  Paid
  #4 Dev      Regular   Salad               $9.00    76  Paid
  #5 Elif     Regular   Burger             $15.00    76  Paid
  #6 Farid    Regular   Pasta              $18.00    76  Waiting for Pasta

Served 5 · Unhappy departures 0 · Revenue $66.00 · Avg sat. 79.2 · Turns 18/18
```

Stage 1 defeat run: result screen

```
DEFEAT
The restaurant fell short of its targets this time.

TARGETS
  ✘ Paid customers                3   at least 4
  ✘ Revenue                  $42.00   at least $45.00
  ✔ Average satisfaction       60.0   at least 60

GUESTS
  Guest       Type      Ordered              Bill  Sat.  Outcome
  #1 Ada      Regular   Salad               $9.00    68  Paid
  #2 Ben      Regular   Burger             $15.00    60  Paid
  #3 Chloe    Regular   Pasta              $18.00    52  Paid
  #4 Dev      Regular   Salad               $9.00    44  Ready to pay
  #5 Elif     Regular   Burger             $15.00    52  Waiting for Burger
  #6 Farid    Regular   -                      -    76  Waiting for a table

Served 4 · Unhappy departures 0 · Revenue $42.00 · Avg sat. 60.0 · Turns 18/18
```

The victory script serves first, then takes orders, then seats, and the Chef cooks straight away: 5 paid, $66.00, average 79.2. The defeat script seats first and the Chef never cooks on the turn an order is taken: 3 paid, $42.00.

7.4 VIP and Critic behaviour (Stage 2)

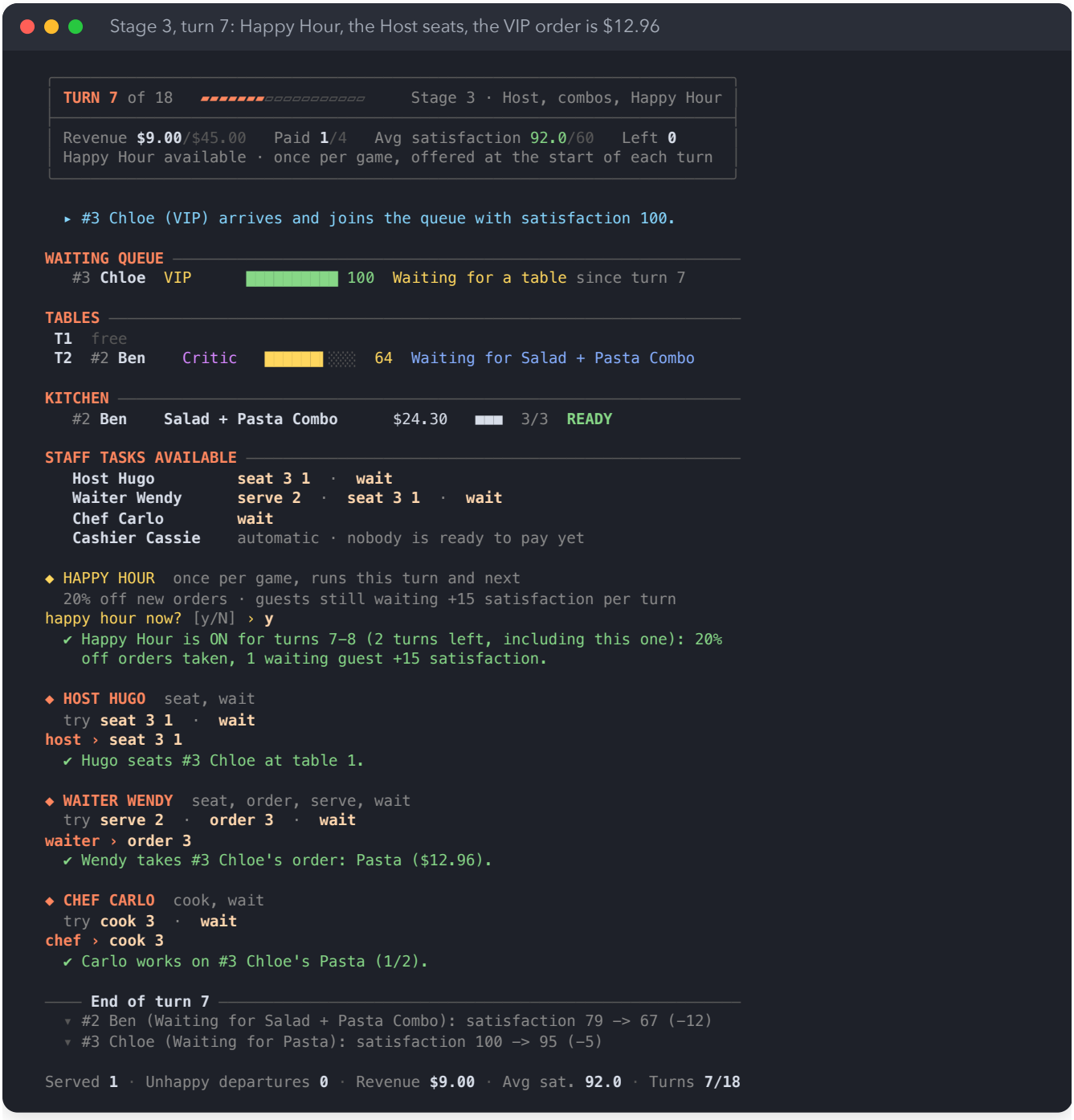
The Critic (#2 Ben) orders Pasta even though the menu rotation would give guest #2 a Burger, and loses 12 points per turn while it cooks. On turn 7 the VIP arrives, and the manager serves the Critic first because the VIP only loses 5 per turn. On turn 9 the Critic pays and leaves a review. On turn 13 the VIP pays **$16.20**.

Stage 2, turn 7: the Critic's Pasta is READY just as the VIP arrives

```
TURN 7 of 18  ▰▰▰▰▰▰▰▱▱▱
```

7.5 Stage 3 features: Host, combos and Happy Hour

On turn 7, Happy Hour is switched on before anyone acts. The Host seats the VIP, and the Waiter takes her order in the same turn ($18.00 × 0.9 × 0.8 = **$12.96**). The Critic is waiting for the 3-unit **Salad + Pasta Combo ($24.30)** and recovers 15 points. On turn 8 the header shows **1 turn left**, and both waiting guests recover again.



Stage 3, turn 8: remaining duration and recovery

```
TURN 8 of 18   ▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱▱▱     Stage 3 · Host, combos, Happy Hour
Revenue $9.00/$45.00   Paid 1/4   Avg satisfaction 92.0/60   Left 0
HAPPY HOUR ON  1 turn left (last turn)  ·  20% off new orders

◆ Happy Hour: #2 Ben perks up to satisfaction 82.
◆ Happy Hour: #3 Chloe perks up to satisfaction 100.
```

7.6 Victory and defeat (Stage 3)

Stage 3 victory run: result screen

```
VICTORY
The dining room hums. Every target was met.

TARGETS
  ✔ Paid customers                5   at least 4
  ✔ Revenue                  $92.16   at least $45.00
  ✔ Average satisfaction       82.6   at least 60

GUESTS
  Guest      Type     Ordered               Bill  Sat.  Outcome
  #1 Ada     Regular  Salad                $9.00    92  Paid
  #2 Ben     Critic   Salad + Pasta Combo $24.30    82  Paid
  #3 Chloe   VIP      Pasta               $12.96    95  Paid
  #4 Dev     Regular  Salad + Burger Combo $21.60   68  Paid
  #5 Elif    Regular  Salad + Pasta Combo $24.30    76  Paid
  #6 Farid   Regular  Salad                $9.00    84  Eating

Served 6 · Unhappy departures 0 · Revenue $92.16 · Avg sat. 82.6 · Turns 18/18
```

Stage 3 defeat run: result screen

```
DEFEAT
The restaurant fell short of its targets this time.

TARGETS
  ✘ Paid customers                3   at least 4
  ✔ Revenue                  $49.50   at least $45.00
  ✘ Average satisfaction       57.0   at least 60

GUESTS
  Guest      Type     Ordered               Bill  Sat.  Outcome
  #1 Ada     Regular  Salad                $9.00    68  Paid
  #2 Ben     Critic   Salad + Pasta Combo $24.30    28  Paid
  #3 Chloe   VIP      Pasta               $16.20    75  Paid
  #4 Dev     Regular  Salad + Burger Combo $17.28   74  Ready to pay
  #5 Elif    Regular  Salad + Pasta Combo $24.30    68  Waiting for food
  #6 Farid   Regular  –                       –     76  Waiting for a table

Served 4 · Unhappy departures 0 · Revenue $49.50 · Avg sat. 57.0 · Turns 18/18
```

The defeat script ignores the Host and switches Happy Hour on at turn 14, too late to help anyone who can still pay: 3 paid, average 57.0.

7.7 Rule checks at each tag

`./test.sh` compiles `src` and `test` with plain `javac`, so no JUnit download is needed. It checks the rules against the real objects, including the brief's worked example and pricing examples, and replays the scripted runs through the real console.

STAGE-1

PASS money is kept in cents and shown with two decimals
PASS worked example 3.3: Pasta from arrival to payment
PASS a dish finished this turn is served next turn at the earliest
PASS an invalid choice explains why and does not use the task
PASS the Waiter completes at most one task per turn
PASS the Cashier takes one payment per turn, oldest first
PASS a customer at 0 leaves, order cancelled, table freed, no revenue
PASS served, eating and paying customers lose no satisfaction
PASS customers choose their own food by rotating through the menu
PASS the game always runs all 18 turns
PASS scripted victory run ends in Victory
PASS scripted defeat run ends in Defeat

12 passed, 0 failed

STAGE-2

PASS money is kept in cents and shown with two decimals
PASS worked example 3.3: Pasta from arrival to payment
PASS a dish finished this turn is served next turn at the earliest
PASS an invalid choice explains why and does not use the task
PASS the Waiter completes at most one task per turn
PASS the Cashier takes one payment per turn, oldest first
PASS a customer at 0 leaves, order cancelled, table freed, no revenue
PASS served, eating and paying customers lose no satisfaction
PASS customers choose their own food by rotating through the menu
PASS the game always runs all 18 turns
PASS VIP: loses 5 per turn and gets 10% off
PASS Critic: loses 12 per turn and orders the most elaborate dish
PASS base setting has six arrivals including a VIP and a Critic
PASS all customer types follow the same order and service flow
PASS scripted victory run ends in Victory
PASS scripted defeat run ends in Defeat

16 passed, 0 failed

STAGE-3

PASS money is kept in cents and shown with two decimals
PASS worked example 3.3: Pasta from arrival to payment
PASS a dish finished this turn is served next turn at the earliest
PASS an invalid choice explains why and does not use the task
PASS the Waiter completes at most one task per turn
PASS the Cashier takes one payment per turn, oldest first
PASS a customer at 0 leaves, order cancelled, table freed, no revenue
PASS served, eating and paying customers lose no satisfaction
PASS customers choose their own food by rotating through the menu
PASS the game always runs all 18 turns
PASS VIP: loses 5 per turn and gets 10% off
PASS Critic: loses 12 per turn and orders the most elaborate dish
PASS base setting has six arrivals including a VIP and a Critic
PASS all customer types follow the same order and service flow
PASS combo: summed units, 90% of the combined price
PASS Host seats one customer in addition to the Waiter
PASS Happy Hour: once per game, this turn and the next
PASS Happy Hour: 20% off after the VIP discount, locked when ordered
PASS Happy Hour: waiting customers recover 15, capped at 100
PASS Happy Hour only starts before any staff member acts
PASS scripted victory run ends in Victory
PASS scripted defeat run ends in Defeat

22 passed, 0 failed

8 Rule interpretations

Where the brief left a choice to us, this is what the game does.

Topic	What the game does
Food choice	A fixed rotation: guest #n orders item n on the menu, wrapping around. The choice is shown when the order is taken.
The Critic's behaviour	Always orders the item with the most preparation units, and the more expensive one on a tie (Pasta in Stage 2, the Salad + Pasta Combo in Stage 3). Leaves a 1 to 5 star review on paying, which does not affect the score.
Combo choice	Combos come after the dishes on the menu, so the rotation gives guests #4 and #5 the two combos.
Combos on the menu	Salad + Burger ($21.60, 2 units) and Salad + Pasta ($24.30, 3 units).
Display order	The scheduled arrival (step 1) happens before the dashboard is drawn, so the manager sees the new customer before choosing.
Host and Waiter order	Host, then Waiter, then Chef. The Waiter may take the order of a customer the Host seated this turn, just as the Chef may cook an order the Waiter took this turn.
Starting Happy Hour	Offered at the start of each turn until used, before any staff member acts. Recovery applies straight away on the first turn and at the start of the second.
"Waiting for a table or food"	`WAITING`, `SEATED` (not yet ordered) and `ORDERED` (including a READY dish not yet served). The same group loses satisfaction at the end of the turn.
Rounding	Half-up to the nearest cent after each percentage: a VIP Burger in Happy Hour goes $15.00, $13.50, $10.80.
Seating	`seat 2` uses the first free table; `seat 2 1` names the table.
Auto-wait (Stage 3)	A staff member whose only possible action is `wait` waits without a prompt.
Recorded satisfaction	A customer's satisfaction is frozen when they pay, so the average uses the values at payment.

9 Reflection and contributions

What worked

- **Planning for all three stages from the start.** Since every stage was published up front, Stage 1 already kept the customer life cycle separate from customer behaviour. Stage 2 then only touched the configuration, one helper in `Money`, and the display.
- **Rules that explain themselves.** Putting each rule in the object that owns the data (`Order.serve`, `Table.assign`, `HappyHour.activate`) gave us clear error messages for free and kept the UI free of rules.
- **Scripts as tests.** Saving runs as plain lists of commands made every victory and defeat reproducible for the marker, the video and the rule checks.

What we would do differently

- Pass `PricingPolicy` and its modifiers in through `GameConfig` instead of creating them in `Game`, so a new policy only needs registering.
- Use structured event objects instead of English strings, so tests can check events instead of text.
- Build the strategy replays from Section 5 into the test suite earlier. They showed that the Host changes the best Waiter priority, which we only noticed late.

Contributions

All four members contributed equally to the project. The table shows the area each member led and the parts of the video they present.

Member	Area led	Share	Video segments
CCDS POLOYAPOY RON JOSHUA ABES (Apollo)	Turn controller (`Game`, `GameConfig`, `ArrivalSchedule`), console UI and command parser	25%	1, 2
CCDS LEONG XUAN HAO	Domain model (`Order`, `Restaurant`, `Table`, `Money`), the kitchen-to-payment flow, VIP and Critic customers	25%	3, 4
CCDS VICTORIA HAUGAA	Stage 3 features (`Host`, `ComboMeal`, Happy Hour) and the scripted victory and defeat runs	25%	5, 6
CCDS LEE SHENG KAI	UML class and sequence diagrams, rule checks, stage comparisons in the README, and the report	25%	7